

# Robot Project Report

CSE2425 Embedded Software

Axel Fouet  
6544843

19.01.2026

## Software Framework

The software is developed in C using the Raspberry Pi Pico SDK. Low-level access to hardware peripherals is achieved through the `hardware` libraries (GPIO, PWM, timers). Several custom drivers are implemented. These drivers are included in the various assignments as needed.

- `motor.c` / `motor.h`: Motor control driver based on PWM generation. The main functions `motor_setup()`, `turn_left()`, and `turn_right()` are used to initialize the motors and control the speed and direction of the left and right wheels.
- `irsensor.c` / `irsensor.h`: Driver for the infrared reflective sensors used for line detection. It provides analog readings for the left and right sensors. The main functions , `irsensor_get_analog_left()`, and `turn_right()`
- `hcsr04.c` / `hcsr04.h`: Ultrasonic sensor driver used for obstacle detection. This module computes the distance to nearby objects using time-of-flight measurements.
- `speed_sensor.c` / `speed_sensor.h`: Wheel encoder driver used for odometry. It measures wheel rotation by counting encoder pulses in order to estimate the traveled distance.

For complex behaviors (Assagnment 3 and 4), the robot logic is structured as a finite state machine (FSM). As described below.

## 1 Motor Control and Basic Motion

The first sub-assignment validates the motor control abstraction. Motors are driven using PWM, where the duty cycle represents the speed percentage. Positive and negative values allow forward and backward rotation, enabling in-place turns.

## 2 Obstacle Detection Algorithm

### 2.1 Principle

Obstacle detection relies on a front-mounted HC-SR04 ultrasonic sensor controlled by the microcontroller through a digital trigger and echo interface. The distance measurement is obtained by actively initiating a measurement sequence and timing the response.

To acquire a distance value, the microcontroller first generates a short trigger pulse on the TRIG pin. The sensor then responds by setting the ECHO pin to a high logic level for a duration proportional to the measured distance. This high-level pulse is captured by the microcontroller using a timer.

## 2.2 Algorithm Description

The algorithm follows a simple reactive strategy:

1. Drive the robot forward at a constant speed
2. Periodically measure the front distance. If the distance is below 15 cm, immediately stop both motors

## 3 Line Detection Algorithm

### 3.1 Sensor Model

Line detection is performed using two analog infrared sensors placed on the left and right sides of the robot. Each sensor outputs a value proportional to surface reflectivity.

A threshold value `LIM_VAL_BLACK` (around 1000. To obtain this value, I placed the sensor on black and white and took a value above the white value.) is used to classify the surface:

- Value above threshold: black line detected
- Value below threshold: white background

### 3.2 Finite State Machine

The line-following algorithm is implemented as a finite state machine with three states:

- `STATE_NO_BLACK` In this state, the robot goes straight ahead.
- `STATE_BLACK_LEFT` In this state, the robot turns right
- `STATE_BLACK_RIGHT` In this state, the robot turns left

### 3.3 Control Logic

In this FSM, all states are connected to each other. If both sensors detect white, the robot moves straight. If the left sensor detects black, the robot turns right; if the right sensor detects black, the robot turns left. If both sensors see black, it remains in the previous state. To prevent the robot from going too fast, it moves in jerks.

## 4 Obstacle Avoidance Algorithm

### 4.1 Overview

Obstacle avoidance combines line following and object tracking using two ultrasonic sensors (front and right side). A multi-state finite state machine (as shown in the figure 1) manages the behavior transitions.

### 4.2 States Description

- `STATE_LINE_FOLLOWER`: Normal line-following mode
- `STATE_AVOID_TURN_LEFT`: Rotation to detect the obstacle with the side sensor. (after this state the robot should be aligned to the object)

- **STATE\_FOLLOW\_OBJECT:** Maintain a constant distance from the obstacle. To do so, if the distance is greater than the target, it turns right; if the distance is smaller, it turns left. To prevent the robot from just turning, I added an epsilon of 4 cm in this range, the robot considers to be a good distance from the object.
- **STATE\_OBJECT\_LOST:** Using the side sensor, it rotates around itself, seeking to minimize the distance seen by the side sensor.
- **STATE\_RETURN\_TO\_LINE:** Return to line-following mode by turning around to find the line

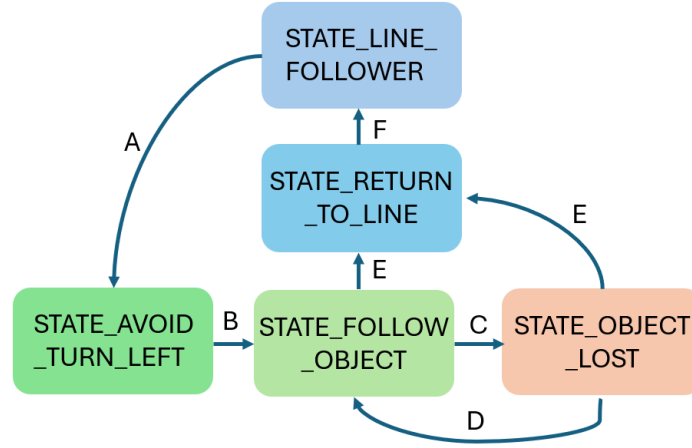


Figure 1: FSM state diagram

### 4.3 Algorithm Logic

**Obstacle detection and initial avoidance (Transition A)** When an obstacle is detected closer than 10cm by the front ultrasonic sensor, the robot immediately stops and switches from STATE\_LINE\_FOLLOWER to STATE\_AVOID\_TURN\_LEFT

**Edge detection and distance calibration (Transition B)** When the robot finds the minimum distance with the side sensor (the robot is aligned with the object), it switches to the state:STATE\_FOLLOW\_OBJECT

**Loss of the obstacle (Transition C and D)** If the side ultrasonic sensor measures a distance larger than 30cm, the obstacle is considered lost. The robot then switches to STATE\_OBJECT\_LOST (transition C). In this state, the robot rotates on the spot while monitoring the side distance. A decreasing then increasing distance pattern is used to detect the obstacle edge again. Once the minimum is detected, the robot returns to STATE\_FOLLOW\_OBJECT (transition D).

**Return to the line (Transitions E and F)** Whenever the line is detected during obstacle following or object search, the robot switches to STATE\_RETURN\_TO\_LINE (transition E). In this state, the robot performs a corrective rotation to realign itself with the line. After a fixed delay, it resumes normal line following by returning to STATE\_LINE\_FOLLOWER (transition F).

## 5 Traveling Distance Calculation

### 5.1 Odometry Model

The traveled distance is estimated using wheel encoders that generate a fixed number of pulses per wheel revolution.

The numbers of pulses is computed as:

$$D = N \cdot \frac{\pi D_w}{P} \quad (1)$$

where  $N$  is the number of pulses,  $D_w$  is the wheel diameter, and  $P$  is the number of pulses per revolution. (I determined this value by turning the wheel and observing how much change was detected. Measured value: 60.)

### 5.2 Algorithm Implementation

A target distance of  $X$  m is converted into a target pulse count. Encoder counters are reset before motion. During operation, left and right encoder counts are read, and the minimum of the two is used to reduce drift. The robot stops once the target pulse count is reached, while continuing line following during motion.

## Conclusion

This project demonstrated the design and implementation of an embedded software architecture for a mobile robot capable of line following, obstacle detection, and obstacle avoidance. Using the Raspberry Pi Pico SDK and low-level C programming, several hardware drivers were developed to control motors, infrared sensors, ultrasonic sensors, and wheel encoders using more or less complex implementations such as interrupts, register manipulation, and PIO programming in assembly.